

WEEK-6

Task-1: Create a client-server application using Named Pipes (FIFOs). The client sends messages and the Server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Code:

```
GNU nano 8.7.1 server.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>

#define REQUEST_FIFO "client_to_server"
#define RESPONSE_FIFO "server_to_client"

int main() {
    char message[100];
    char response[150];

    mkfifo(REQUEST_FIFO, 0666);
    mkfifo(RESPONSE_FIFO, 0666);

    printf("Server started...\n");

    int read_fd = open(REQUEST_FIFO, O_RDONLY);
    int write_fd = open(RESPONSE_FIFO, O_WRONLY);

    while (1) {
        memset(message, 0, sizeof(message));

        read(read_fd, message, sizeof(message));

        printf("Client: %s\n", message);

        if (strcmp(message, "exit") == 0)
            break;

        snprintf(response, sizeof(response),
                 "Server received: %s", message);

        write(write_fd, response, strlen(response) + 1);
    }
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify

```
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define REQUEST_FIFO "client_to_server"
#define RESPONSE_FIFO "server_to_client"

int main() {
    char message[100];
    char response[150];

    int write_fd = open(REQUEST_FIFO, O_WRONLY);
    int read_fd = open(RESPONSE_FIFO, O_RDONLY);

    while (1) {
        printf("Enter message: ");
        fflush(stdout);

        fgets(message, sizeof(message), stdin);
        message[strlen(message)] = '\0';

        write(write_fd, message, strlen(message) + 1);

        if (strcmp(message, "exit") == 0)
            break;

        memset(response, 0, sizeof(response));

        read(read_fd, response, sizeof(response));

        printf("Server: %s\n", response);
    }

    close(write_fd);
    close(read_fd);
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
**^\
^** Replace

^K Cut
^U Paste

^T Execute
^J Justify

Output:

```
the_boss@DESKTOP-I8C8AJG:~/practical$ nano server.c
the_boss@DESKTOP-I8C8AJG:~/practical$ nano client.c
the_boss@DESKTOP-I8C8AJG:~/practical$ gcc server.c -o server
the_boss@DESKTOP-I8C8AJG:~/practical$ gcc client.c -o client
the_boss@DESKTOP-I8C8AJG:~/practical$ ./server
Server started...
Client: Hello Server
Client: exit
Server stopped.
the_boss@DESKTOP-I8C8AJG:~/practical$ |
```

Report: The client-server application was successfully implemented using Named Pipes in Ubuntu.

The client sends messages through a FIFO and the server receives and processes them.

The server sends a response back through another FIFO.

Multiple-client behavior was also tested, showing that clients share the common communication channel.

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1.

Demonstrate asynchronous event handling and explain the role of signal handlers.

Code:

```
GNU nano 8.7.1                                signal.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig) {
    if (sig == SIGINT)
        printf("\nSIGINT received!\n");
    else if (sig == SIGTERM)
        printf("\nSIGTERM received!\n");
    else if (sig == SIGUSR1)
        printf("\nSIGUSR1 received!\n");

    fflush(stdout);
}

int main() {
    signal(SIGINT, handler);
    signal(SIGTERM, handler);
    signal(SIGUSR1, handler);

    printf("Process ID: %d\n", getpid());
    printf("Waiting for signals...\n");

    while (1) {
        printf("Program is running...\n");
        sleep(3);
    }

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

Output:

```
SIGUSR1 received!
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...

SIGUSR1 received!
Program is running...
Program is running...
Program is running...
Program is running...
Killed                                ./signal
the_boss@DESKTOP-I8C8AJG:~/practical$
the_boss@DESKTOP-I8C8AJG:~/practical$ |
```

Report: The POSIX signal handling program was successfully implemented in Ubuntu. The program handled SIGINT, SIGTERM, and SIGUSR1 using signal handlers. Signals were generated using Ctrl+C and the kill command. The program demonstrated asynchronous event handling while continuing its normal execution.